

Rubric

1 Overview

Rubric converts a job description into an explicit scoring rubric, then evaluates every applicant against that rubric and nothing else. Applicants submit a resume and a two minute voice introduction. Both are scored criterion by criterion, with every point traceable to a verbatim quote from a named source. Shortlisted candidates complete an adaptive voice interview whose questions are planned for rubric coverage and worded from what the candidate actually said.

The design principle throughout: **the model never emits a bare total**. It emits per-criterion sub-scores that are summed and verified in code before any row is written. A single holistic number drifts 15 to 20 points between runs on identical input; anchored sub-scores hold steady. Every score in the product is reproducible and explainable.

2 Scope

In scope

- Job creation and automatic rubric generation
- Resume upload with text extraction
- In-browser voice introduction capture
- Dual-source screening against the rubric
- Ranked HR dashboard and candidate review
- Manual HR approval gate
- Adaptive voice interview, 5 to 10 questions
- Interview evaluation and result review
- Nine screens, two distinct user experiences

Out of scope **FUTURE MODULES**

- Webcam proctoring, face and gaze detection
- Anti-cheat and integrity scoring
- GitHub and LinkedIn verification
- PDF report generation
- Automated email delivery
- HR authentication and multi-user access
- Deployment, hosting, custom domain
- Rubric editing by hand

3 Functional requirements

3.1 Job definition and rubric

ID	REQUIREMENT
FR-1.1	HR creates a job with title, description, required skills and experience level.
FR-1.2	The system generates a rubric of 4 to 7 named criteria with integer point allocations summing to exactly 100.
FR-1.3	A rubric whose points do not total 100 is regenerated with the violation stated, never silently corrected.
FR-1.4	The generated rubric is displayed to HR immediately on creation, with per-criterion descriptions.
FR-1.5	HR can regenerate a rubric, with a warning if applicants already exist.
FR-1.6	Each job exposes a copyable public application link.

3.2 Application and screening

ID	REQUIREMENT
FR-2.1	A candidate submits name, email, a resume PDF up to 5 MB, and a voice introduction recorded in the browser.
FR-2.2	Resume text is extracted locally. An image-only PDF is rejected with instructions rather than accepted as empty text.
FR-2.3	The introduction is transcribed by a hosted model, falling back to a local model on rate limit, server error or timeout.
FR-2.4	Resume text and transcript are scored against the same rubric in a single evaluation.
FR-2.5	Each criterion receives points and a list of verbatim evidence quotes, each tagged as originating from the resume or the introduction.
FR-2.6	Sub-scores are verified in code to sum to the reported total. A mismatch fails validation.
FR-2.7	Every evidence quote is verified to appear in the source it names. A quote attributed to the resume that exists only in the transcript fails validation.
FR-2.8	Discrepancies between resume and introduction are recorded and shown to HR as neutral observations. They carry no score penalty.
FR-2.9	Matched and unevicenced skills are listed separately. Uneviced means neither source mentioned it, not that the candidate lacks it.
FR-2.10	A recommendation band is computed server-side and returned with the score. The frontend never derives a band from a number.

3.3 Review and approval

ID	REQUIREMENT
FR-3.1	HR sees all applicants for a job ranked by screening score, with recommendation and pipeline status.
FR-3.2	The list can be filtered by recommendation band.
FR-3.3	Candidate detail shows the score, the per-criterion breakdown with source-tagged evidence, the resume, the transcript, and audio playback.
FR-3.4	Approving a candidate mints a unique, opaque, non-guessable interview token and produces a copyable link.
FR-3.5	Rejecting a candidate requires explicit confirmation.
FR-3.6	A failed screening can be retried without the candidate reapplying.

3.4 Adaptive interview

ID	REQUIREMENT
FR-4.1	The interview is reachable only by its token and only once, before completion.

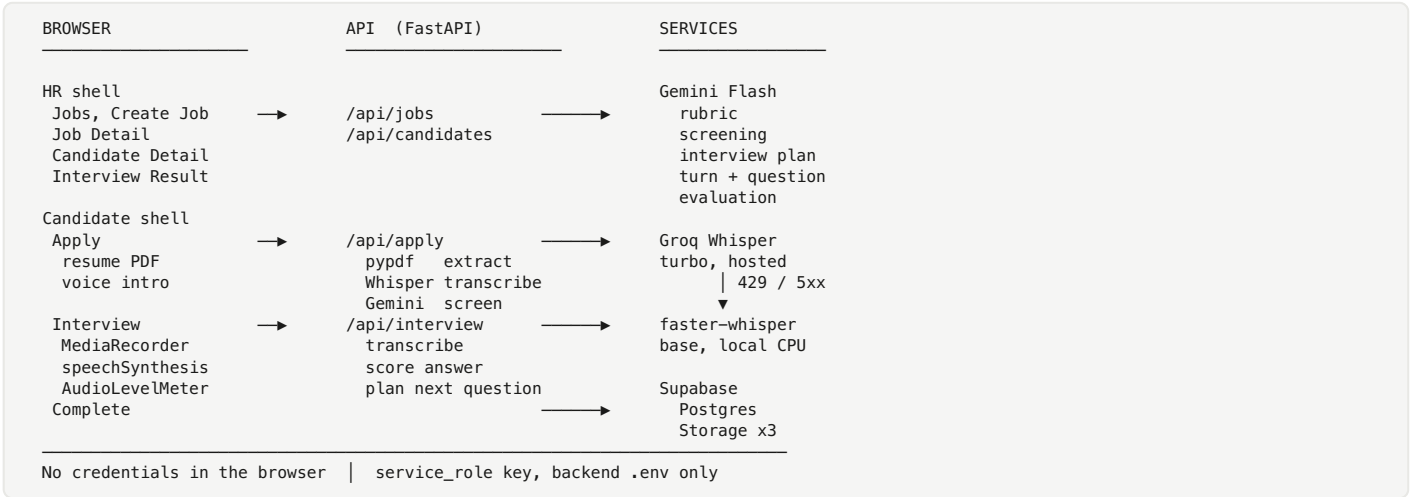
ID	REQUIREMENT
FR-4.2	A question plan is generated before the first question: 5 to 10 slots, every rubric criterion covered at least once, difficulty progressing from opening to deep.
FR-4.3	The first three slots have fixed intents: background, projects, personal contribution.
FR-4.4	Questions are spoken aloud by the browser and remain visible as text throughout.
FR-4.5	Recording starts automatically once the question is delivered. The candidate is never asked to press record.
FR-4.6	Each answer is transcribed and scored against its slot's criteria as it arrives, not deferred to the end.
FR-4.7	Each subsequent question is generated from carried interview state, targeting an uncovered criterion and anchored on a specific claim the candidate made.
FR-4.8	A question substantially similar to any earlier question is regenerated before being asked.
FR-4.9	Processing displays the real backend stage. No percentage bars, no timer-driven fake progress.
FR-4.10	Refreshing mid-interview resumes at the correct question. Answers are persisted per turn.
FR-4.11	An answer that fails transcription can be re-recorded in the same slot without restarting.
FR-4.12	The candidate is never shown a score, band, recommendation or evaluation.

3.5 Evaluation

ID	REQUIREMENT
FR-5.1	Technical, communication and experience sub-scores are computed in code from accumulated per-answer scores. The model does not invent them.
FR-5.2	The overall score is a weighted average, verified within tolerance before being written.
FR-5.3	Strengths and concerns each reference something the candidate actually said.
FR-5.4	HR sees the full transcript with the rubric criteria each question probed and the response time per answer.

4 System architecture

Three tiers with a single trust boundary. The browser never holds a credential for any external service; every call to the database, the language model or the transcription service originates from the API layer.



Frontend

React with Vite, plain CSS over a token layer. Two shells that share tokens but nothing else.

HR shell	Sidebar application. Dense tables, scannable, repeat use.
Candidate shell	Centered, no navigation. Single-use, one task at a time.
Interview	Full viewport, no chrome at all. One control on screen.

Notable components. VoiceRecorder wraps native MediaRecorder and handles both Chrome webm/opus and Safari mp4 explicitly. AudioLevelMeter is driven by a real AnalyserNode, never a simulated waveform, because it is the candidate's only proof the microphone is working. FileDropzone validates type and size before upload. ScoreBreakdown and EvidenceList render per-criterion points with their source-tagged quotes.

Text to speech uses the browser's own speechSynthesis. It costs nothing, needs no key, and has no network round trip, so questions begin immediately rather than after an API call.

5 Data model

TABLE	HOLDS	NOTABLE COLUMNS
jobs	Role definition and its rubric	rubric jsonb, state

Backend

FastAPI with Pydantic v2. Fully synchronous: no queue, no workers, no background tasks. Screening holds the request open for 8 to 20 seconds, which is correct at this scale and removes an entire class of failure.

llm.py	Structured Gemini calls, retry with violation stated
stt.py	Hosted transcription, local fallback
resume.py	PDF extraction and normalisation
validation.py	Post-generation checks for every stage
interview.py	Plan, state object, turn advancement
scoring.py	Band computation, weighted aggregation
heuristics.py	Every threshold as a named constant
cassettes.py	Offline replay for demonstrations

Every numeric threshold in the system lives in one file. No literal with meaning appears anywhere else.

TABLE	HOLDS	NOTABLE COLUMNS
candidates	Applicant, both sources, screening outcome	resume_text, transcript, sub_scores jsonb, band, state
interviews	Token, question plan, running state	plan jsonb, state_object jsonb
interview_turns	One row per question and answer	slot, criterion_ids, response time
interview_results	Final evaluation	overall and three sub-scores, strengths, concerns

Three private storage buckets hold resumes, introduction audio and answer audio. The database stores object *paths*, never URLs; the API resolves a path to a signed URL at response time. Row level security stays enabled and the backend bypasses it with a service role key held only in the server environment.

6 AI pipeline

Five calls, each returning a validated schema. No free-text parsing anywhere in the system.

STAGE	INPUT	OUTPUT	VALIDATED ON
Rubric	Job description, skills, experience	4 to 7 criteria with points	Points total exactly 100
Screening	Rubric, resume text, transcript	Sub-scores, evidence, conflicts, recommendation	Sum matches total; every quote found in its named source
Plan	Rubric, screening, transcript	5 to 10 planned slots	Every criterion covered; depth progresses
Turn	Answer, plan slot, interview state	Answer score, claims, next question	Question not similar to any prior; criteria exist
Evaluation	Accumulated scores, transcript	Strengths, concerns, recommendation	Overall within tolerance of weighted average

THE INTERVIEW STATE OBJECT

Carried into every question generation call: questions already asked, answers given, topics discussed, claims made, criteria covered, criteria remaining, and depth reached per topic. The plan guarantees the interview covers the whole rubric. The state guarantees each question is specific. Without the plan, an interview follows whatever the candidate mentioned first and leaves most criteria unprobed. Without the state, follow-ups are generic. Together they produce *"how did you handle the cold-start problem in that recommender"* instead of *"tell me about your experience with system design."*

7 Non-functional requirements

AREA	REQUIREMENT
Cost	Free service tiers only, including during development. No paid API call at any point.
Deployment	Localhost only. No hosting, no domain.
Reliability	An offline replay mode reproduces a full demonstration from recorded responses, verified with the network disconnected.
Consistency	A test harness scores one fixture five times and asserts score range stays under threshold. Variance is the only measurable property where no ground truth exists.
Interview quality	A synthetic candidate plays through a full interview and a second pass grades the interviewer on repetition, rubric coverage, anchoring and depth progression.
Accessibility	Semantic HTML, visible focus throughout, keyboard navigable, no colour-only signals, reduced-motion respected, usable at 375 px.
Privacy	Audio and resumes in private buckets behind signed URLs. Candidates see no scores. Academic demonstration; not for screening real applicants.
Concurrency	Verified for 5 to 10 simultaneous candidates. No claim is made beyond that.

8 Delivery

PHASE	WORK	DAYS
0	Database, storage, API skeleton, token layer	1
1	Transcription with fallback, resume extraction	1.5
2	Rubric generation and validation	1
3	Dual-source screening, consistency harness	3
4	Interview plan, state, turns, evaluation	2
5	Design foundation, primitives, both shells	1.5
6	Interview screen, recorder, level meter	2
7	Application screen with resume upload	1.5
8	Four HR screens	2.5
9	Landing and completion screens	0.5
10	Offline replay, hardening, handover	1.5

15 to 17 working days. Work is ordered to retire risk before building surface: the AI stages and the audio pipeline are built and verified from the command line first. A complete end-to-end flow runs from a terminal at roughly day six; the first visual demonstration lands around day nine.

Two gates are absolute. Frontend work does not begin until the score consistency harness passes, because a polished interface over drifting scores is worth less than a plain one over stable scores. Nothing is handed over until offline replay has been verified with the network physically disconnected.